

Dataset Preparation Guide

Task 2: Staffing Needs Prediction

Tech-Triathlon Competition

Document Purpose: This guide provides a complete step-by-step approach to prepare your datasets for predicting staffing needs using machine learning. Follow each phase sequentially to transform raw data into a model-ready training dataset.

PHASE 1: Understanding Your Raw Data

Available Datasets

1. Bookings Dataset

Contains information about every booking made at the government office.

Column Name	Data Type	Description
booking_id	String	Unique booking reference number
citizen_id	String	Anonymized ID of the citizen
appointment_date	Date	Date of the appointment
task_id	String	Links to Tasks Dataset
check_in_time	DateTime	Actual check-in time
check_out_time	DateTime	Actual check-out time

2. Tasks Dataset

Maps tasks to their corresponding sections in the government office.

Column Name	Data Type	Description
task_id	String	Unique task identifier
task_name	String	Descriptive name (you fill this)
section_id	String	Section handling this task
section_name	String	Section name (you fill this)

3. Staffing Dataset

Shows daily staffing levels and productive task time for each section.

Column Name	Data Type	Description
date	Date	Date of staffing record

section_id	String	Section identifier
employees_on_duty	Integer	Number of employees (TARGET)
total_task_time_minutes	Integer	Total productive minutes

⚠ **Key Challenge:** At prediction time, you will only have `section_id` and `date` as inputs. All features must be derivable from these two inputs using historical patterns.

PHASE 2: Step-by-Step Data Preparation

1 Load and Explore Each Dataset

Begin by loading each dataset and understanding its structure. Check for:

- Missing values
- Data types
- Date ranges covered
- Number of unique sections (should be 6)
- Number of unique tasks per section

Sample Data Structure

```
Bookings Dataset Sample: booking_id | citizen_id | appointment_date | task_id | check_in_time |  
check_out_time BK001 | CIT123 | 2024-07-15 | T001 | 09:00:00 | 09:25:00 BK002 | CIT124 | 2024-07-  
15 | T001 | 09:05:00 | 09:28:00 BK003 | CIT125 | 2024-07-15 | T005 | 09:10:00 | 09:45:00
```

2 Calculate Processing Times

Add a new column to calculate actual service time for each booking:

```
processing_minutes = (check_out_time - check_in_time) in minutes
```

Result

booking_id	appointment_date	task_id	processing_minutes
BK001	2024-07-15	T001	25
BK002	2024-07-15	T001	23
BK003	2024-07-15	T005	35

3 Join Bookings with Tasks

Link bookings to their respective sections using the task_id:

```
JOIN bookings WITH tasks ON bookings.task_id = tasks.task_id
```

Result

booking_id	appointment_date	task_id	section_id	processing_minutes
BK001	2024-07-15	T001	SEC_1	25
BK002	2024-07-15	T001	SEC_1	23
BK003	2024-07-15	T005	SEC_2	35

4 Aggregate Daily Statistics Per Section

Group bookings by date and section to create daily summaries:

Aggregation Operations

- **num_bookings:** COUNT of bookings per section per day
- **avg_processing_time:** AVERAGE processing time
- **total_processing:** SUM of all processing times
- **peak_hour:** Hour with most check-ins

Daily Section Summary

date	section_id	num_bookings	avg_processing_time	total_processing
2024-07-15	SEC_1	45	24	1080
2024-07-15	SEC_2	28	32	896
2024-07-16	SEC_1	38	23	874

5 Create Base Training Dataset

Merge the aggregated data with the staffing dataset to create your base training data:

```
MERGE daily_statistics WITH staffing_dataset ON date = date AND section_id = section_id
```

Base Training Dataset

date	section_id	num_bookings	avg_processing	employees_on_duty	total_task_time
2024-07-15	SEC_1	45	24	6	380
2024-07-15	SEC_2	28	32	4	245

✓ **Checkpoint:** You now have historical data showing what actually happened each day - how many bookings occurred, how long they took, and how many staff were used.

6 Engineer Features

Create features that can be calculated from just date + section_id:

6.1 Temporal Features (from date)

Feature	Description	Value Range
day_of_week	Monday=0, Sunday=6	0-6
day_of_month	Day of the month	1-31
month	Month number	1-12
quarter	Quarter of year	1-4
week_of_year	Week number	1-52
is_weekend	Saturday/Sunday flag	0 or 1
is_month_start	First 5 days of month	0 or 1
is_month_end	Last 5 days of month	0 or 1

6.2 Historical Pattern Features

Pre-calculate these from training data:

Feature	Calculation Method
avg_bookings_this_weekday_section	Average bookings for this section on this weekday
avg_staff_this_weekday_section	Average staff for this section on this weekday
median_processing_time_section	Median task time for this section
std_bookings_this_weekday	Standard deviation of bookings (variability indicator)

6.3 Rolling Window Features

Calculate from recent historical data:

Feature	Window	Description
avg_bookings_last_7_days	7 days	Recent booking trend
avg_staff_last_7_days	7 days	Recent staffing levels
avg_bookings_last_30_days	30 days	Monthly trend
booking_trend	14 days	Increasing/decreasing/stable

6.4 Section-Specific Features

section_id	avg_daily_bookings	avg_daily_staff	complexity_score
SEC_1	42.3	5.5	0.7
SEC_2	31.2	4.2	0.9
SEC_3	55.6	7.1	0.5

7 Create Final Training Dataset

Combine all features into your final training dataset:

Final Dataset Structure

FEATURES (X)					TARGET (y)
date	section_id	day_of_week	hist_avg_bookings	rolling_avg_7d	employees_on_duty
2024-07-15	SEC_1	1	44.5	42.3	6
2024-07-16	SEC_1	2	38.2	41.8	5
2024-07-15	SEC_2	1	27.8	28.1	4

PHASE 3: Prepare for Model Training

8 Split Your Data

⚠ **Important:** Use time-based splitting for time series data!

Recommended Split Strategy

- **Training Set (80%):** January 2024 - September 2024
- **Validation Set (20%):** October 2024 - December 2024
- **Sort by date** before splitting to maintain temporal order
- **Never use random splitting** for time series data

9 Create Feature Generation Pipeline

Build a function that can generate all features from just section_id and date:

```
Input: (section_id="SEC_1", date="2025-09-15") ↓ Feature Generation Pipeline: 1. Extract temporal features from date 2. Lookup historical averages for section+weekday 3. Calculate rolling statistics 4. Retrieve section-specific features ↓ Output: Feature vector ready for model
```

Feature Generation Example

Step	Operation	Result
1	Extract day_of_week from date	1 (Monday)
2	Lookup historical average for SEC_1 + Monday	44.5 bookings
3	Calculate days since last holiday	14 days
4	Retrieve section complexity score	0.7

